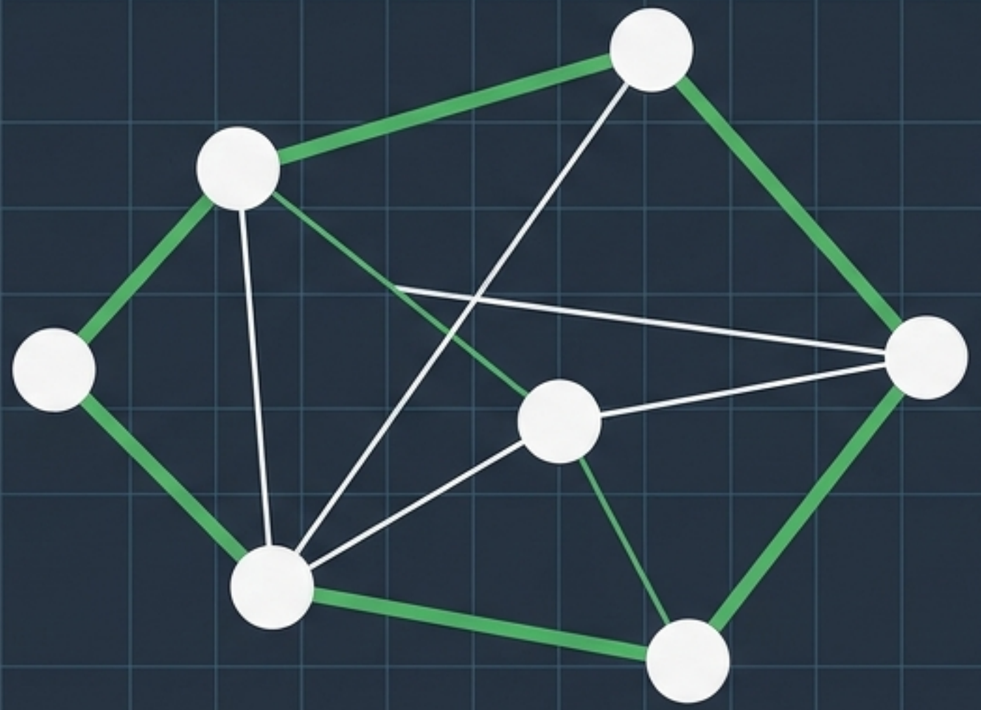
Kruskal's Algorithm: Building the Minimum Spanning Tree

A Greedy Approach to Optimal Networks.

EDGE	WEIGHT		STATUS
U	V	W	ACCEPTED
1	2	3	YES
2	3	4	YES
1	4	5	YES
4	5	6	YES
3	6	7	YES
5	6	8	NO
1	3	9	NO
5	7	10	YES



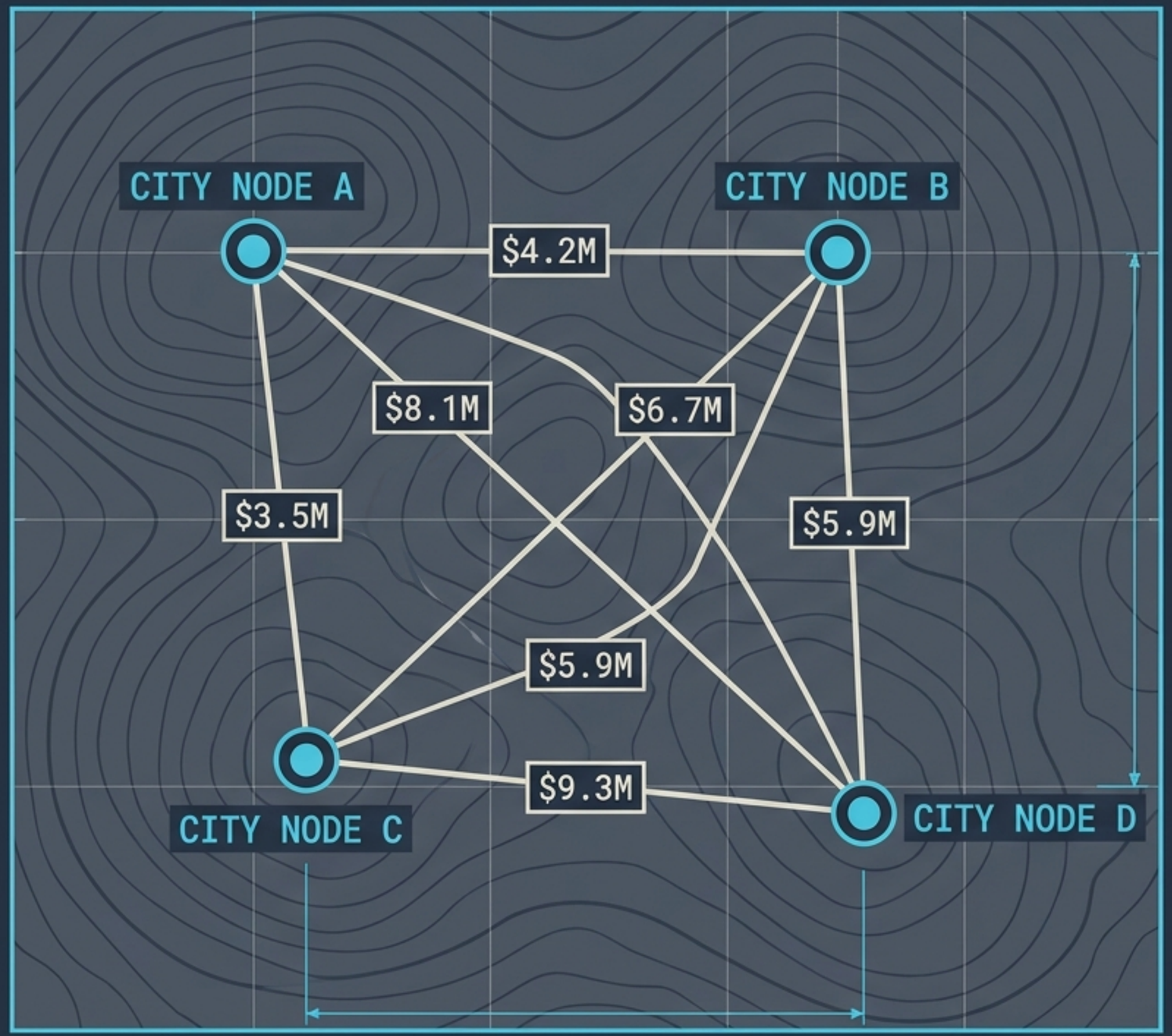
STEP 1: LIST EDGES ASCENDING ORDER (W).
STEP 2: ITERATE THROUGH SORTED EDGES.
STEP 3: ADD EDGE IF NO CYCLE IS FORMED.
FOCUS:

ADD EDGE(1,2) W=3 [CYCLE: NO].
ADD EDGE(2,3) W=4 [CYCLE: NO].
ADD EDGE(1,4) W=5 [CYCLE: NO].
ADD EDGE(4,5) W=6 [CYCLE: NO].
ADD EDGE(3,6) W=7 [CYCLE: NO].
EDGE(5,6) W=8 [CYCLE: YES, REJECT].
EDGE(1,3) W=9 [CYCLE: YES, REJECT].
ADD EDGE(5,7) W=10 [CYCLE: NO].

MST COMPLETE.

Created using NotebookLM

Gemini Notebook

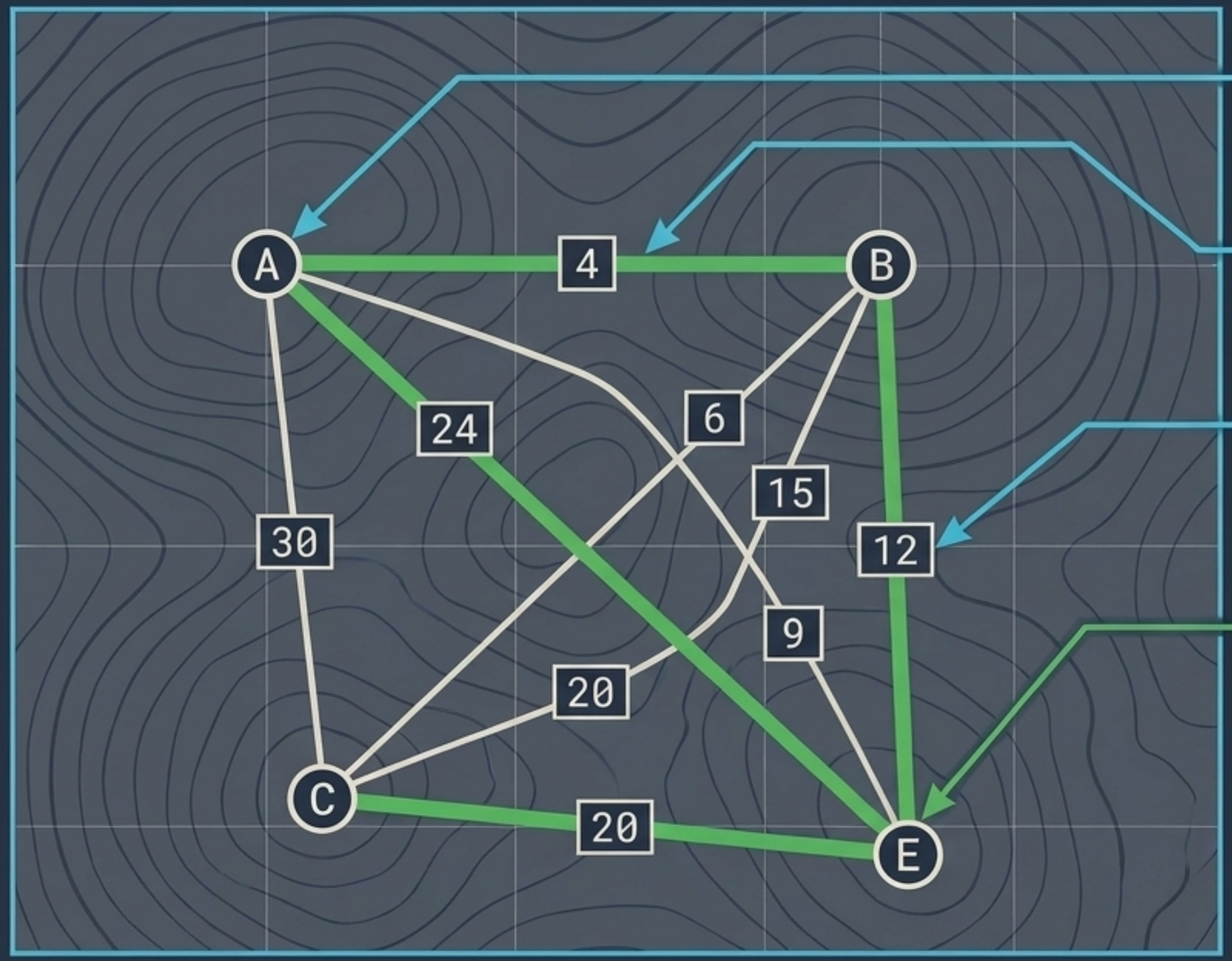


THE CORE INFRASTRUCTURE PROBLEM

How do you connect multiple locations with the absolute minimum infrastructure cost?

KRUSKAL'S ALGORITHM IS THE DEFINITIVE SOLUTION FOR FINDING THE CHEAPEST PATH THAT CONNECTS EVERYTHING WITHOUT REDUNDANT LOOPS.

Gemini Notebook



Vertex (V): A point or location in the graph.

Edge (E): A connection between two vertices.

Weight: The cost or value of using the edge.

Minimum Spanning Tree (MST): Connects every vertex, contains zero cycles, and has the absolute smallest total edge weight.

Rule of Math: For a graph with V vertices, the spanning tree always uses exactly V - 1 edges.

Gemini Notebook

The Greedy Logic: Sort, Select, Check



SORT

Order all edges by weight, from cheapest to most expensive.



SELECT

Always evaluate the absolute cheapest available edge first.

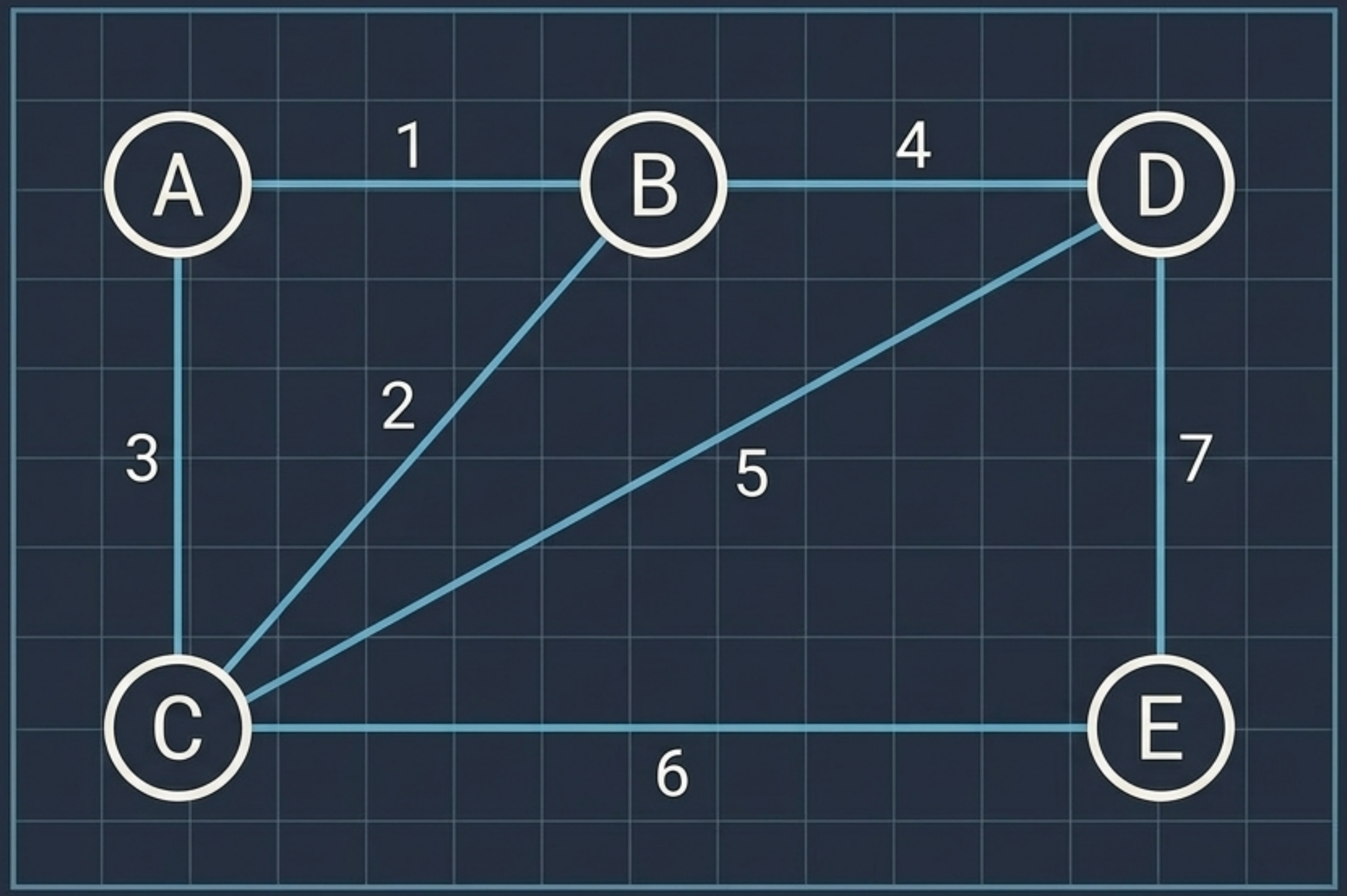


CHECK

Accept the edge only if it connects new components. Reject if it creates a closed cycle.

Gemini Notebook

The Setup: 5 Vertices, 7 Potential Edges

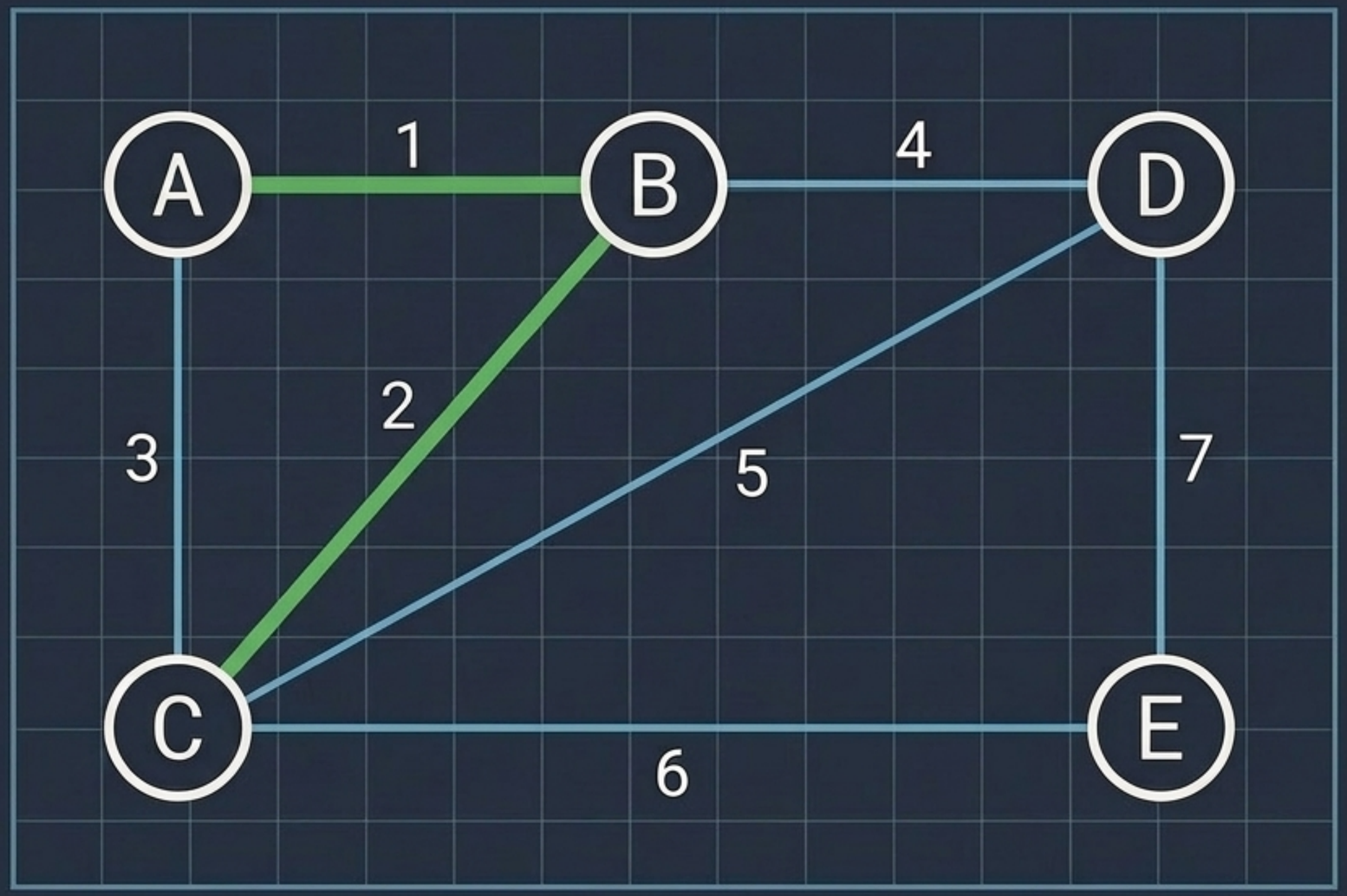


The Edge Ledger	
Edge	Weight
A–B	(Weight: 1)
B–C	(Weight: 2)
A–C	(Weight: 3)
B–D	(Weight: 4)
C–D	(Weight: 5)
C–E	(Weight: 6)
D–E	(Weight: 7)

Goal: Connect 5 vertices (Requires V−1 = 4 edges).

Gemini Notebook

Steps 1 & 2: The Safe Bets



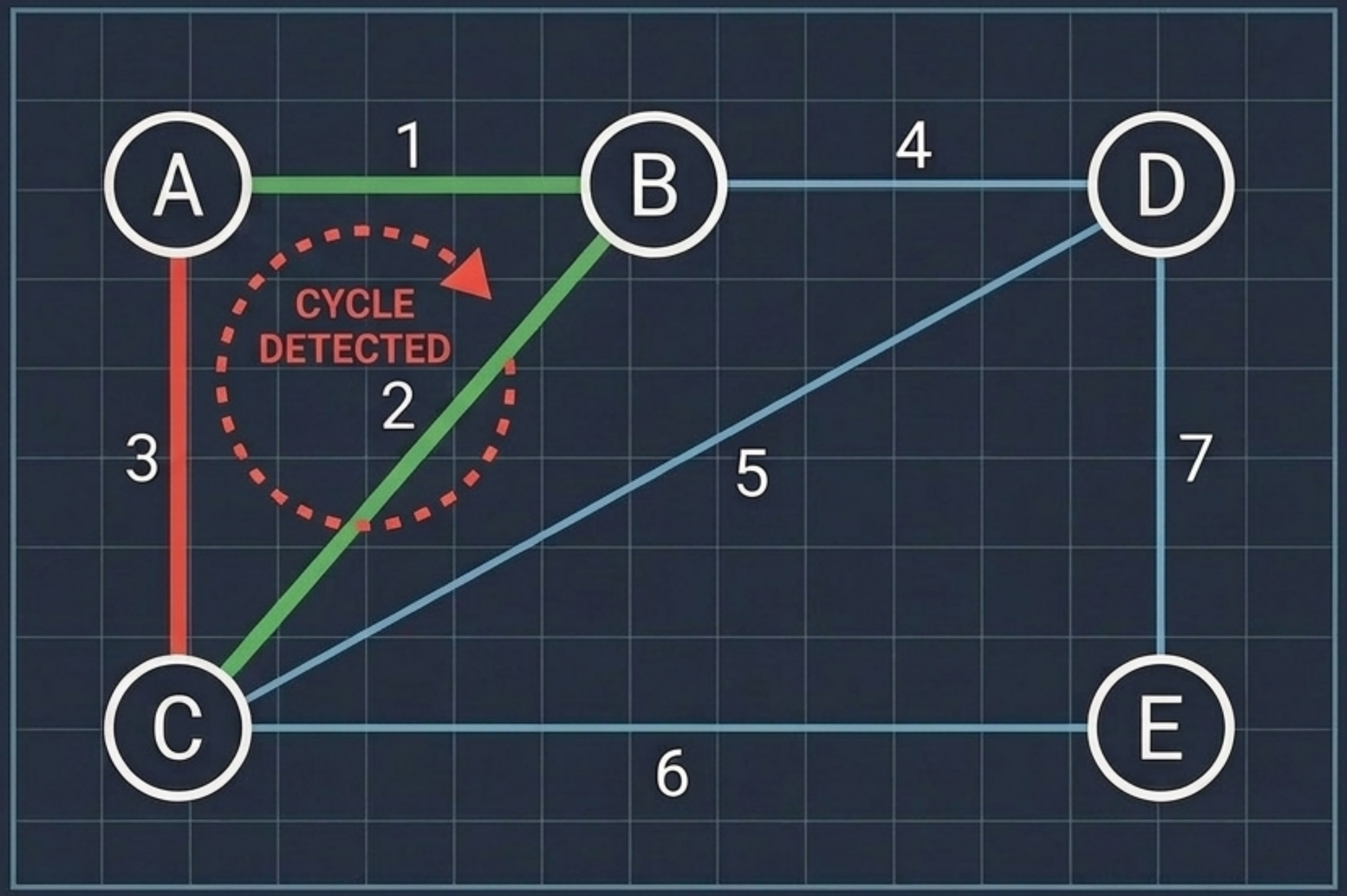
Current MST Edges: 2 / 4

The Edge Ledger	
A–B	(Weight: 1) ✓
B–C	(Weight: 2) ✓
A–C	(Weight: 3)
B–D	(Weight: 4)
C–D	(Weight: 5)
C–E	(Weight: 6)
D–E	(Weight: 7)

Because the graph is mostly empty, the first two cheapest edges easily connect isolated vertices without any risk of forming a cycle. Accepted.

Gemini Notebook

Step 3: The Cycle Threat



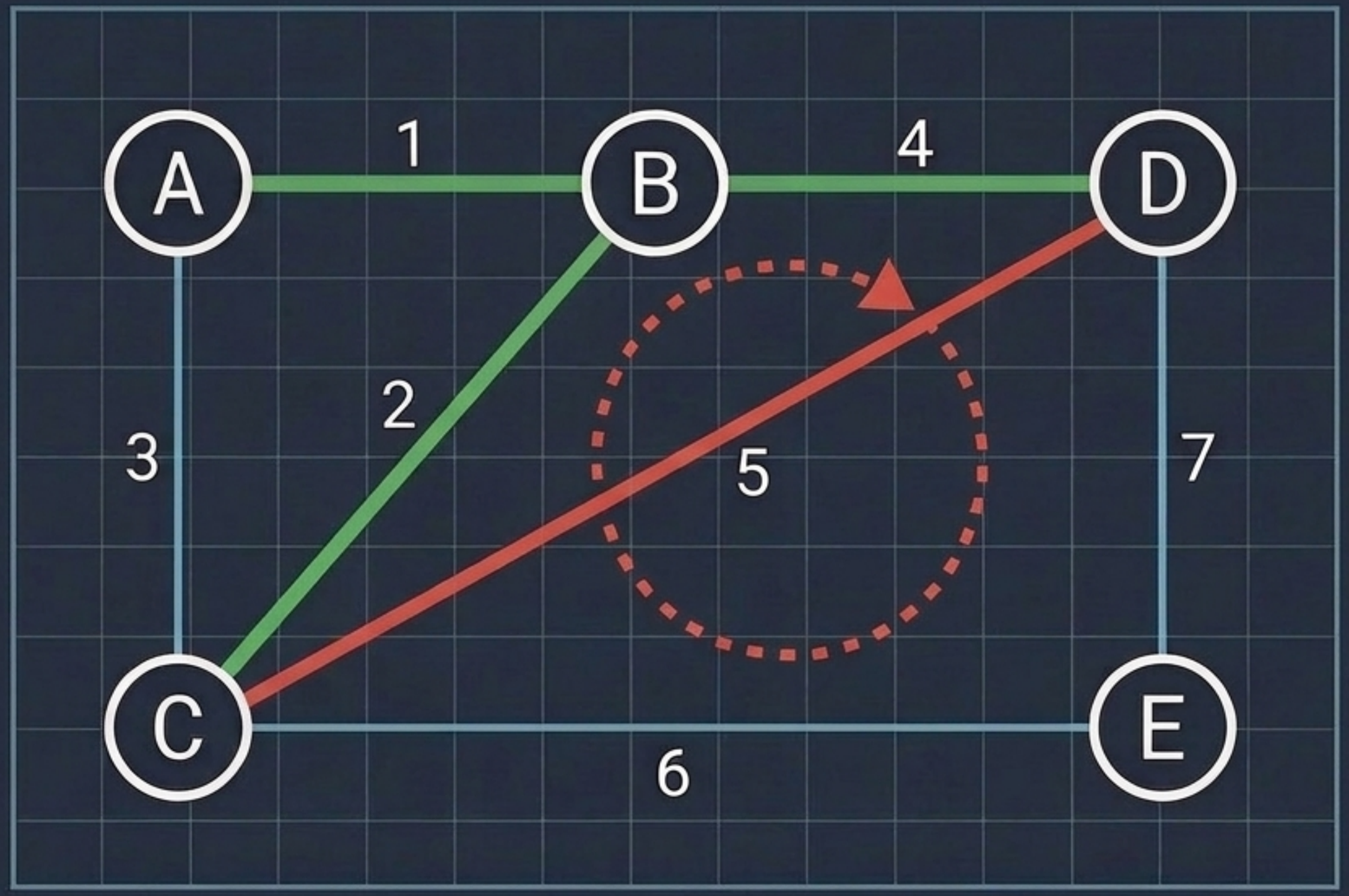
The Edge Ledger	
A–B	(Weight: 1) ✓
B–C	(Weight: 2) ✓
~~A–C~~	~~(Weight: 3)~~ ✗
B–D	(Weight: 4)
C–D	(Weight: 5)
C–E	(Weight: 6)
D–E	(Weight: 7)

REJECTED. A and C are already connected through vertex B. Adding edge A–C would form a closed loop (A–B–C–A). The greedy rule mandates skipping this edge.

Current MST Edges: 2 / 4

Gemini Notebook

Steps 4 & 5: Expansion and Rejection



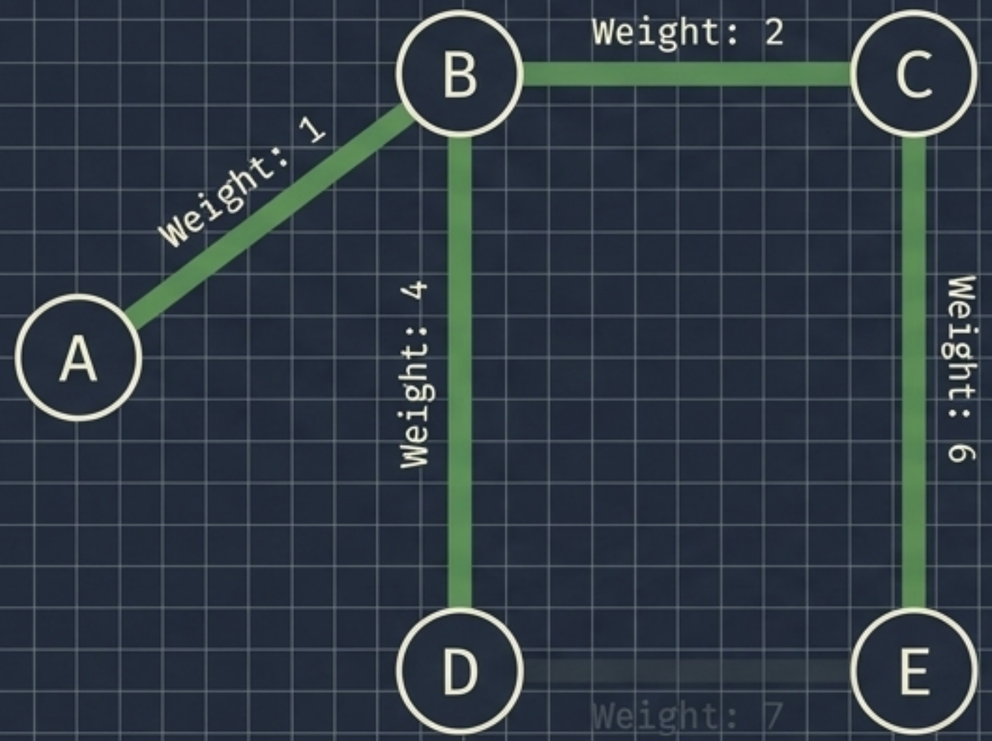
Current MST Edges: 3 / 4

The Edge Ledger	
A–B	(Weight: 1) ✓
B–C	(Weight: 2) ✓
~~A–C~~	~~(Weight: 3)~~ ✗
B–D	(Weight: 4) ✓
~~C–D~~	~~(Weight: 5)~~ ✗
C–E	(Weight: 6)
D–E	(Weight: 7)

- B–D (4) **ACCEPTED**. Connects new vertex D.
- C–D (5) **REJECTED**. C and D are already part of the same connected network via B.

Gemini Notebook

Step 6: The Final Link and Termination



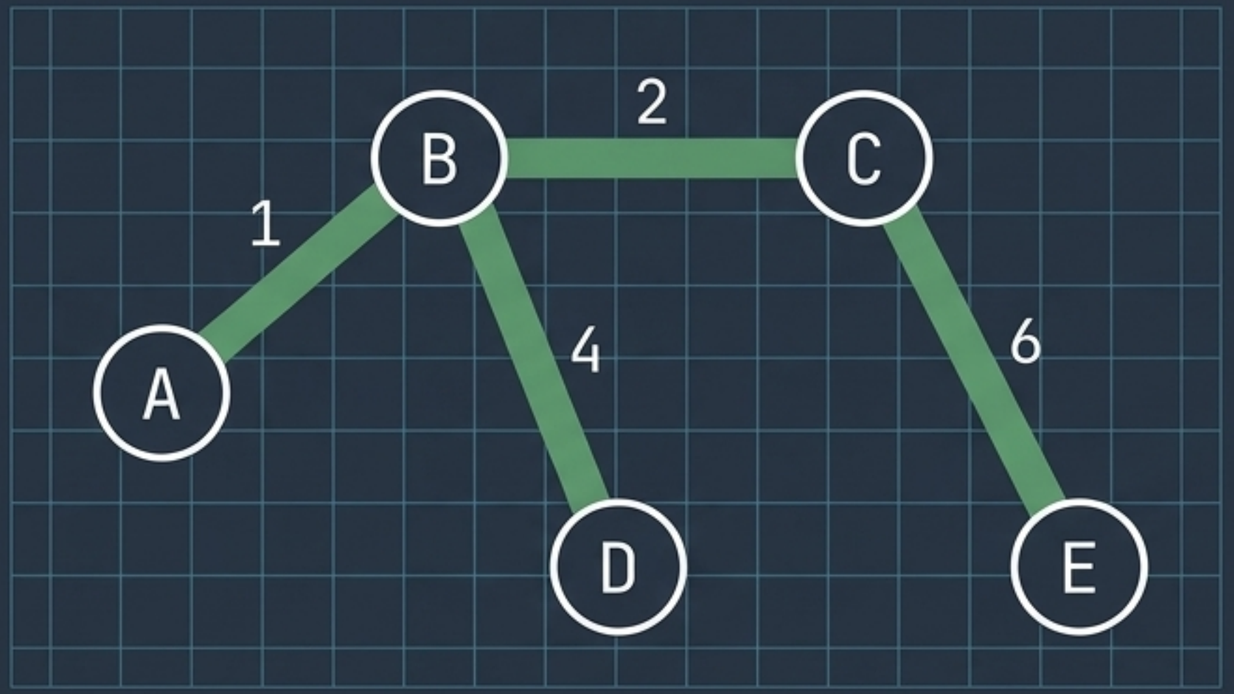
The Edge Ledger		
1	A–B (Weight: 1)	✓
2	B–C (Weight: 2)	✓
3	~~A–C (Weight: 3)~~	✗
4	B–D (Weight: 4)	✓
5	~~C–D (Weight: 5)~~	✗
6	C–E (Weight: 6)	✓
7	D–E (Weight: 7)	STOP

HALT. Edge C–E connects our final vertex, E. We have now selected exactly 4 edges (V - 1). The algorithm stops immediately. D–E is never evaluated.

Current MST Edges: 4 / 4 (Complete)

Gemini Notebook

The Completed Minimum Spanning Tree



Total MST Weight = 1 + 2 + 4 + 6 = 13

Vertices connected: 5	Edges used: 4 (Exactly V - 1)
Cycles created: 0	Total Cost: 13 (The mathematical minimum)

Gemini Notebook

The Blueprint: Kruskal's Pseudocode

```
1. Sort all edges in increasing order of weight.
2. Initialize an empty Minimum Spanning Tree.
3. For each edge in sorted order:
4.     If adding the edge does not create a cycle:
5.         Add to MST.
6.     Otherwise, reject the edge.
7. Stop when MST contains V - 1 edges.
```

SORT

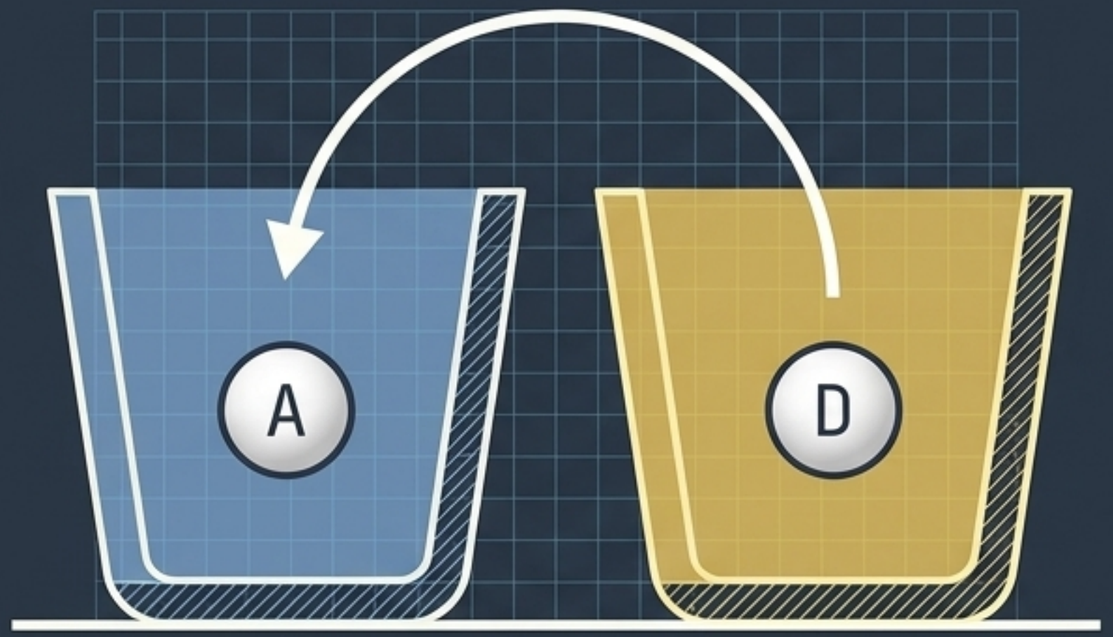
CHECK

HALT

Gemini Notebook

Visualizing the Cycle Detector: Disjoint Set Union (DSU)

Union(u, v)



Merges two different components into one network.

Find(u) == Find(v)



If Find(u) equals Find(v), they are already in the same bucket. Adding an edge equals a guaranteed cycle!

Gemini Notebook

Performance Profile: Time Complexity

The Efficient Check (Union-Find)

Detecting cycles takes nearly constant time, meaning it barely impacts the overall speed.

$$O(E \log E)$$

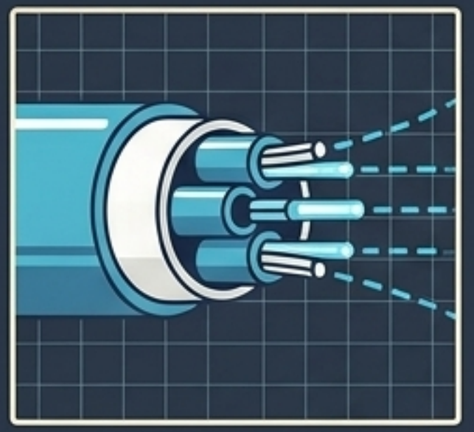
The Dominant Cost

Sorting all edges by weight is the heaviest computational lifting the algorithm performs.

E = Total number of edges | V = Total number of vertices

Gemini Notebook

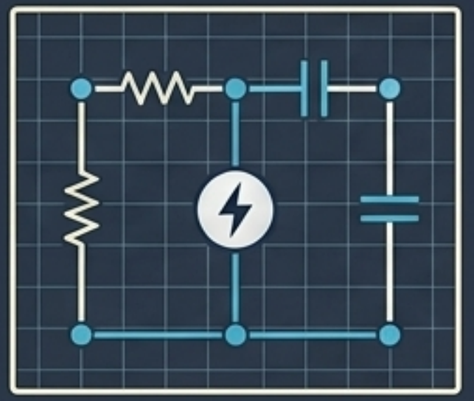
Designing the Real World: Kruskal's Applications



Designing cost-effective communication networks.



Connecting municipal infrastructure with minimal build costs.



Optimizing electrical wiring and power grids.



Mapping efficient computer network topology.

Used globally wherever multiple locations must be connected at the absolute minimum resource cost.

Gemini Notebook

The Master Cheat Sheet

1

SORT

Order all edges by increasing weight.

(Sorting dominates runtime: $O(E \log E)$)

2

SELECT

Pick the cheapest edge available that connects different components.

(Use Union-Find to group networks)

3

AVOID CYCLES

Stop immediately when you have selected exactly $V - 1$ edges.

A greedy approach to connecting every vertex at minimum total cost.

Gemini Notebook